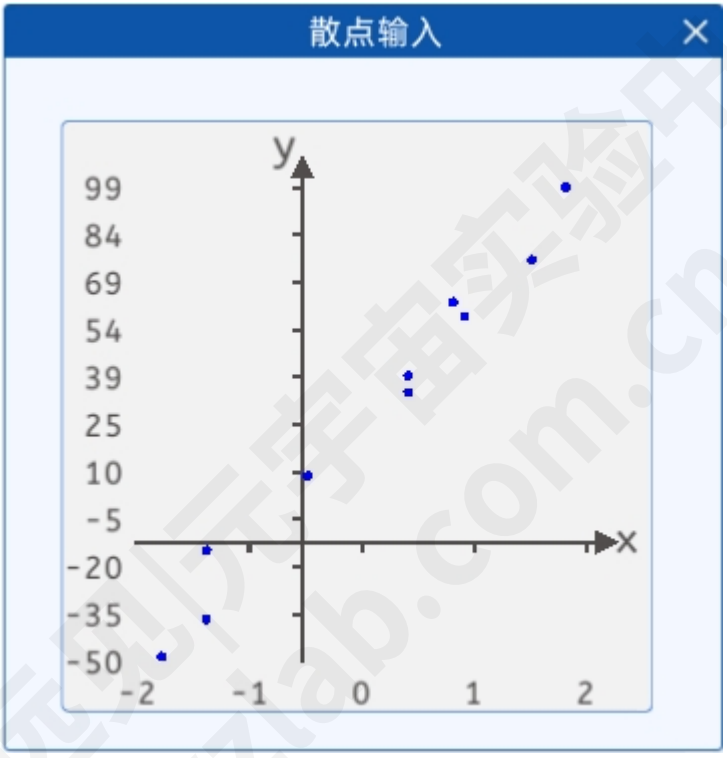
优化器与优化方法

1. 基础原理

1.1 散点输入

本实验准备了一些散点，其分布如下图所示：

其坐标为：[-0.5, 7.7]，[1.8, 98.5]，[0.9, 57.8]，[0.4, 39.2]，[-1.4, -15.7]，[-1.4, -37.3]，[-1.8, -49.1]，[1.5, 75.6]，[0.4, 34.0]，[0.8, 62.3]



现在希望根据这些已有的散点来拟合一条直线，来描述这些散点之间的关系，并通过该直线能预测一些新的点的坐标。

1.2 参数初始化

由于最终的目标是拟合一条线，所以还是使用 $y = wx + b$ 这个表达式，并通过“参数初始化”这个组件来初始化 w 和 b 两个参数，用来进行第一次前向计算。

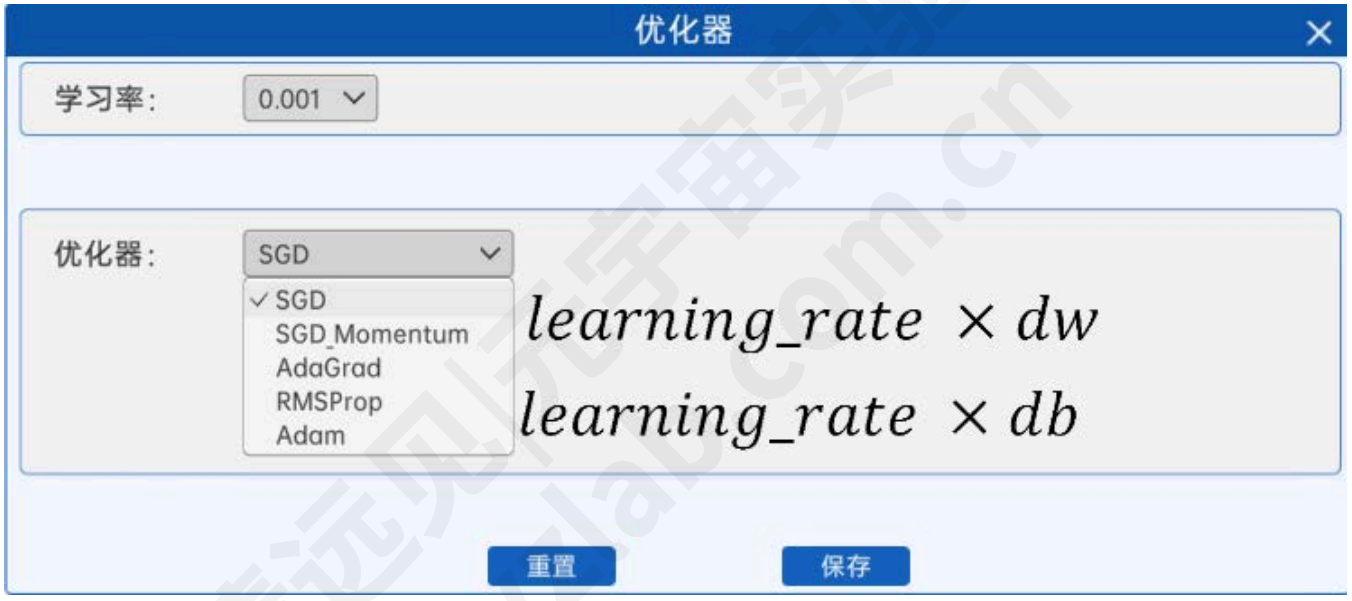
1.3 损失函数

由于是线的拟合，所以我们这里选择的损失函数是MSE（均方差损失函数）。

1.4 优化器

在之前的实验中，我们大多数用的是**Gradient Descent**（梯度下降法）这个优化方法，也有用到过SGD（随机梯度下降，Stochastic Gradient Descent）。

在本实验中的“优化器”组件中，共有5种优化器，如下图所示：



不同的优化器有着各自的优缺点，下面来一个个介绍这些优化器：

1.4.1 SGD

梯度下降法有批量梯度下降（Batch Gradient Descent）、随机梯度下降（Stochastic Gradient Descent）和小批量梯度下降（Mini-batch Gradient Descent）。

1. **批量梯度下降**

批量梯度下降指的是它在每一次迭代中都使用全部的训练数据来更新模型参数。

由于它的全部数据都一起进行梯度下降，所以相对更加**高效**和**可以朝着全局最优解进行。**

但是由于是全部数据一起进行梯度下降，在大规模数据集上内存或者显存会不够，造成**极大的计算开销。**

2. **随机量梯度下降**

它是另一个极端，批量梯度下降是将全部数据集一起进行梯度下降，它是只拿出一个样本进行梯度下降，由于数据少，所以**计算开销**很小，但是**效率不是很高。**

3. **小批量梯度下降**

它是前两者的一个各取所长的方案，将训练数据集成若干个小批量，进行梯度下降，并根据该样本的梯度来更新参数的值。这种方案取得了相对平衡，由于每次取多少是可以被提前确定的，所以可以根据计算机性能进行选择。

在实际的深度学习中，虽然叫SGD，看似是随机梯度下降，但是却使用的是小批量梯度下降。可以把他们统称为SGD，很多时候（博客上）叫它随机梯度下降或者小批量梯度下降是一样的，但是需要注意到它实际上不是一个东西。

然而，由于使用的是样本梯度来更新参数，随机梯度下降的更新方向较为不稳定，可能会出现震荡现象，导致无法收敛到全局最优解，尤其是在非凸函数中。

> 使用SGD生成代码，查看其内部运行规则。

1.4.2 SGD_Momentum

SGD+Momentum是随机梯度下降（Stochastic Gradient Descent，SGD）的一种变体，它引入了动量（Momentum）的概念来改进传统的随机梯度下降算法。动量的引入有助于解决SGD中的震荡和收敛速度较慢的问题。可以更好地应对梯度变化和梯度消失问题，从而提高训练模型的效率和稳定性。

更新规则（TensorFlow框架）：

$$\text{动量计算：} v_t = \eta \cdot v_{t-1} + \alpha \cdot g_t$$

$$\text{优化结果计算：} P_{t+1} = P_t - v_t$$

其中：

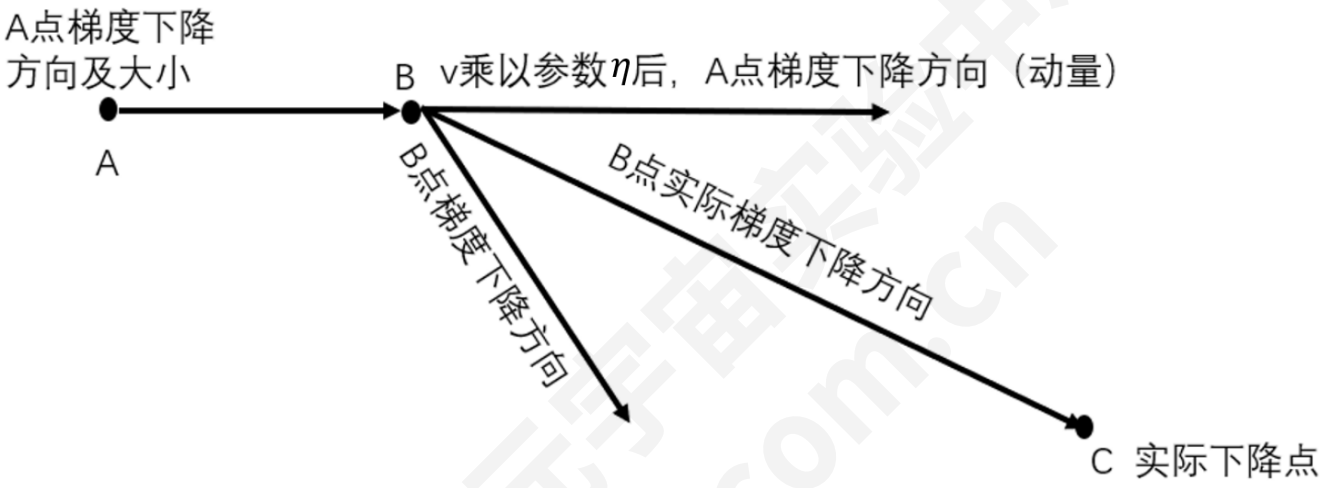
- α 是学习率
- g_t 是在t时刻 W_t或者b_t 这些参数的损失梯度

- η 是动量系数，处于0到1之间，值越大，说明越重视上一次的影响，为零时就是SGD，通常设置为0.9
- v_t 是动量（Momentum），它是一个滑动平均值，用于积累梯度的方向
- v_{t-1} 初始值是0

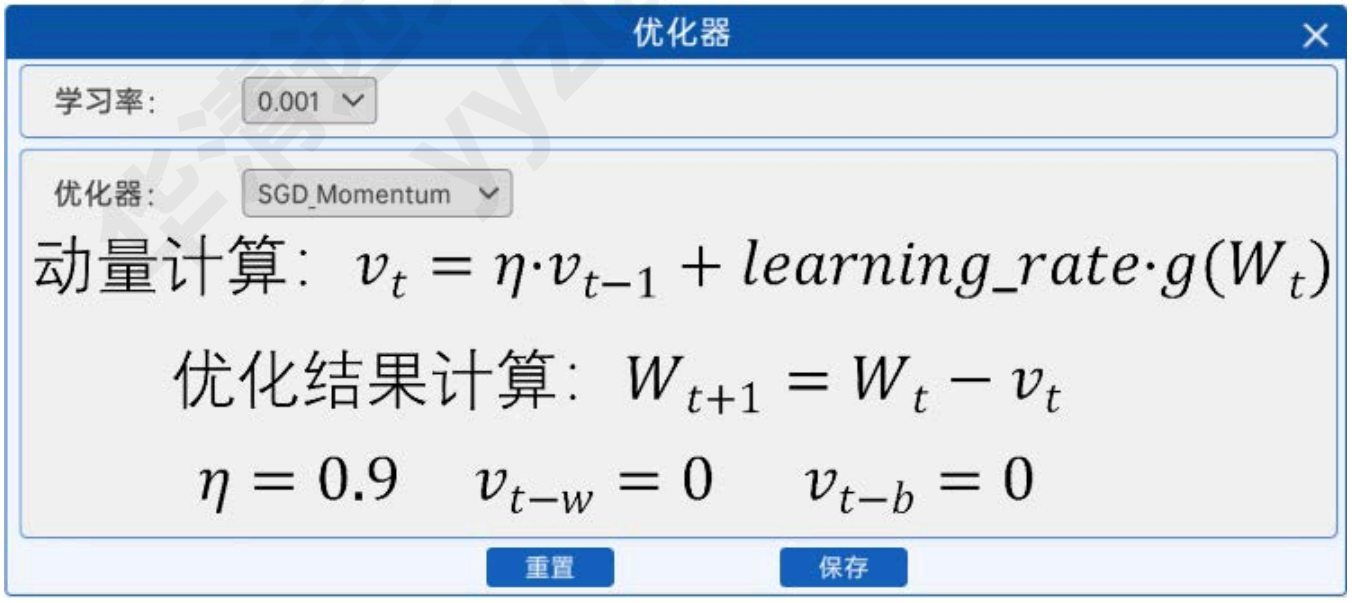
它的工作方式是在每次迭代时都会计算模型参数的梯度，然后将梯度值乘以一个因子（称为动量或动量系数），再将乘积累加到模型参数中。

这样，模型参数的更新不再仅仅依赖于当前梯度值，还依赖于过去的梯度值，有助于**加速收敛**。同时，动量有助于减小更新的方差，从而**减小震荡**，使得参数更新更加平滑。

具体如下图所示：



在组件中，显示了其中一些参数的初始化值，其中，表示动量系数，表示为w更新时的动量值，表示为b更新时的动量值，如下图所示：



> 使用SGD+Momentum生成代码，查看其内部运行规则。

在PyTorch中，SGD+Momentum更新规则不同：

$$\text{动量计算：} v_t = \eta \cdot v_{t-1} + g_t$$

$$优化结果计算：P_{t+1} = P_t - \alpha \cdot v_t$$

其中：

- α 是学习率
- g_t 是在t时刻 W_t或者b_t 这些参数的损失梯度
- η 是动量系数，处于0到1之间，值越大，说明越重视上一次的影响，为零时就是SGD，通常设置为0.9
- v_t 是动量（Momentum），它是一个滑动平均值，用于积累梯度的方向
- v_{t-1} 初始值是第一步的梯度值

1.4.3 AdaGrad

上面两种方法的学习率都是固定的，而Adagrad引入了二阶动量，它可以是一种自适应学习率优化器，通过对每个参数使用不同的学习率来解决学习率固定的问题。

SGD以及动量以同样的学习率更新每个参数。神经网络模型往往包含大量的参数，但是这些参数并不会总是用得到，更新频率也不一样。对于经常更新的参数，不希望其被单个样本影响太大，希望学习速率慢一些；对于偶尔更新的参数，了解的信息太少，希望能够从每个偶然出现的样本身上多学一些，即学习速率大一些。

在训练过程中，Adagrad会根据每个参数的梯度值来自动调整它们的学习率，使得每个参数都能够适当地更新。

怎么去度量历史更新频率呢？就是二阶动量，它的更新方案为：

$$S_t = S_{t-1} + g_t \cdot g_t$$

$$P_{t+1} = P_t - \frac{\alpha}{\sqrt{S_t + \varepsilon}} \cdot g_t$$

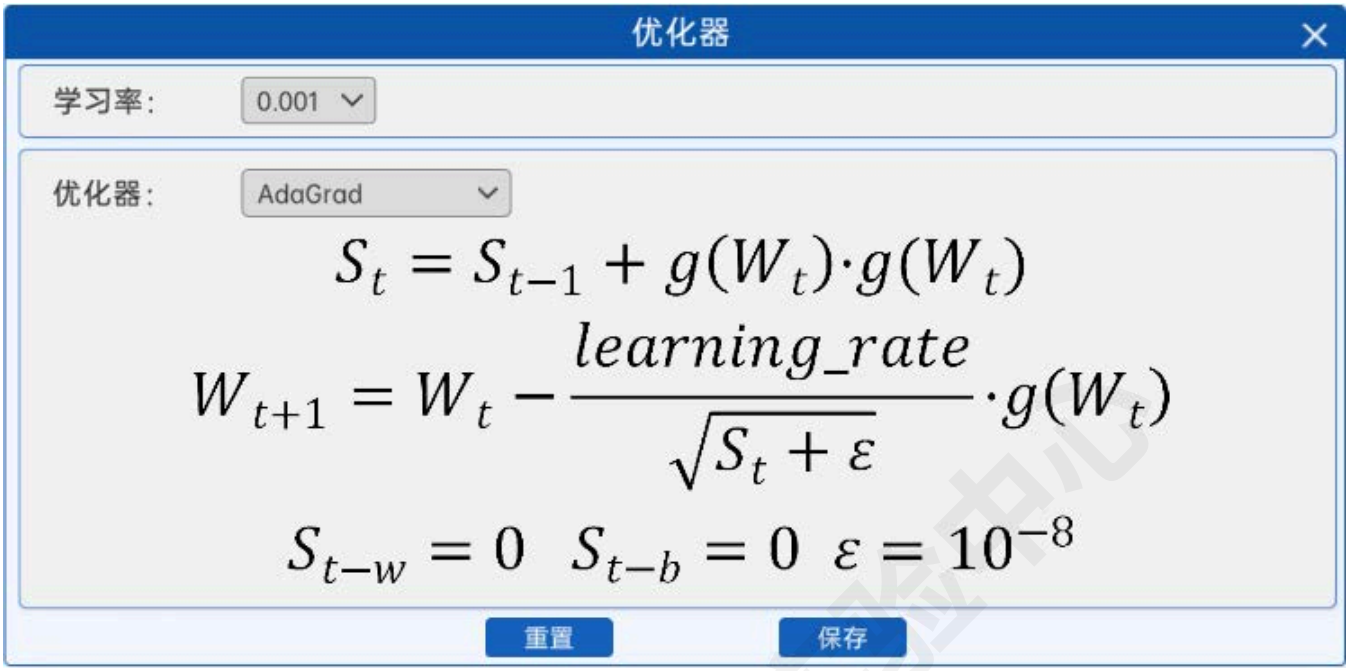
其中：

- α 是学习率
- g_t 是在t时刻 W_t或者b_t 这些参数的损失梯度
- ε 是为了防止分母为0设置的，一般为10^{-7}
- S_t 其实就是对损失梯度平方求和

优点：它可以让参数在训练的前期更新得快一些，在后期更新得慢一些，有助于**稳定训练**，从而**更快地收敛到最优值**，不同更新频率的参数具有不同的学习率，减少摆动，在稀疏数据场景下表现会非常好。

缺点：但是由于分母中是历史梯度平方的累积和，随着训练的进行，分母会逐渐增大，导致**学习率递减得很快**，可能造成训练提前停止。

在本实验中，表示w更新时用的损失梯度平方和，表示b更新时用的损失梯度平方和，如下图所示：



> 使用AdaGrad生成代码，查看其内部运行规则。

1.4.4 RMSProp

为了解决Adagrad优化器的缺点，Geoff Hinton提出了RMSProp优化器，其实相比于Adagrad优化器，RMSProp优化器只是添加了一个系数用来控制衰减速度，它的更新方案为：

$$S_t = \eta \cdot S_{t-1} + (1-\eta) \cdot g_t \cdot g_t$$

$$P_{t+1} = P_t - \frac{\alpha}{\sqrt{S_t + \varepsilon}} \cdot g_t$$

其中：

- α 是学习率
- g_t 是在t时刻 W_t或者b_t 这些参数的损失梯度
- ε 是为了防止分母为0设置的，一般为10^{-7}
- η 为控制衰减速度参数，一般为0.9

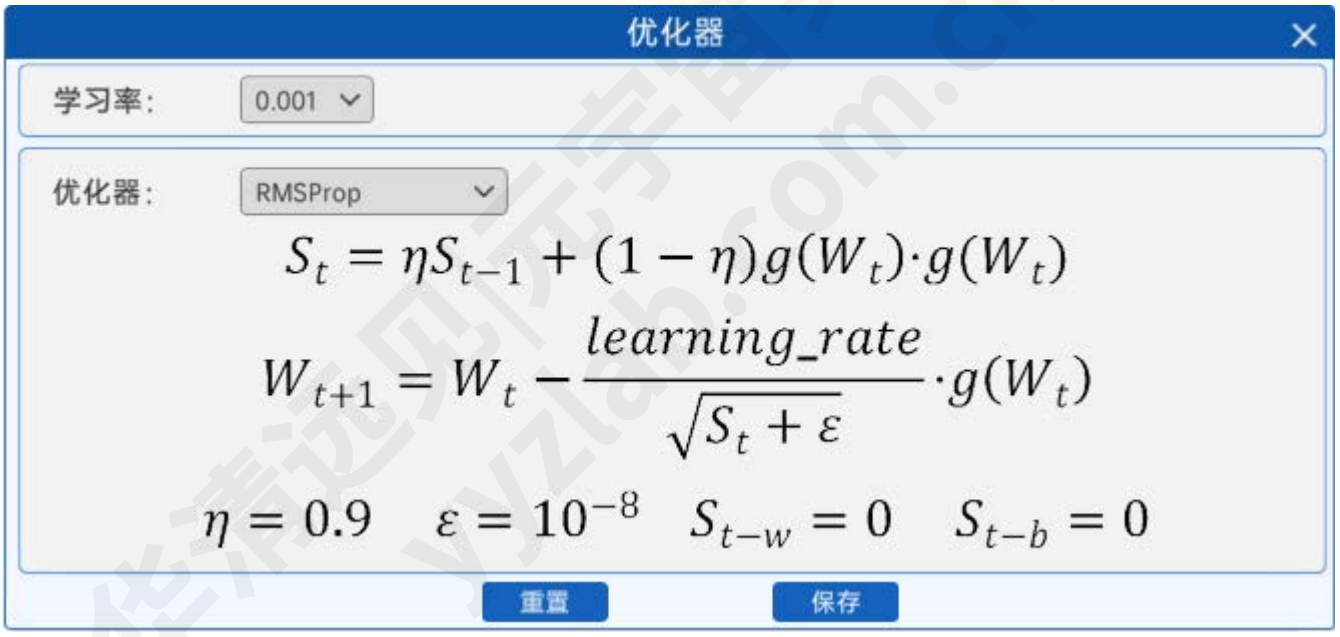
> **指数加权移动平均（Exponential Moving Average，EMA）**：
> 指数加权移动平均是一种对时间序列数据进行平滑的方法。在指数加权移动平均中，过去的观测值被赋予不同的权重，而且随着时间的推移，权重呈指数递减。这意味着离当前时刻越远的观测值权重越小。
>
> 例如：$S_t = \eta \cdot S_{t-1} + (1-\eta) \cdot g_t \cdot g_t$

> 默认使 $\eta = 0.9$，那么：
> $S_{100} = 0.9 \cdot S_{99} + 0.1 \cdot g_{100}^2$
> $S_{99} = 0.9 \cdot S_{98} + 0.1 \cdot g_{99}^2$
> $S_{989} = 0.9 \cdot S_{97} + 0.1 \cdot g_{98}^2$
>
> 带入到总公式中：
> $S_{100} = 0.1 \cdot g_{100}^2 + 0.1 \cdot 0.9 \cdot g_{99}^2 + 0.1 \cdot 0.9^2 \cdot g_{98}^2 \ldots + 0.1 \cdot 0.9^{99} \cdot g_1^2$
>
> 第100个指数加权移动平均的值包含99的、98的、97的，而且是一个指数衰减的过程。

优点：RMSProp的思想是在更新参数的同时，使用梯度的指数加权平方的移动平均来调整学习率，克服了AdaGrad中**学习率递减过快**的问题。这个算法通过不断调整学习率来帮助模型更快地收敛。RMSProp与Adagrad算法类似，但它**更加稳定**，并且**更容易调参**。

但是，类似于AdaGrad，RMSProp仍然未考虑梯度的方向，这导致在某些情况下**学习率的适应性可能不足。**

在本实验中，S_{t-w}表示w更新时用的损失梯度平方和，S_{t-b}表表示b更新时用的损失梯度平方和。



> 使用RMSProp生成代码，查看其内部运行规则。

1.4.5 Adam

1. **矩估计引入**

Adam(Adaptive Moment Estimation)自适应矩估计，也是一种自适应学习率的优化器，它结合了Momentum和RMSProp两种优化方法的算法，引入了两个新参数 β_1 和 β_2，其一阶动量和二阶动量公式为：

$$\text{一阶矩估计：} m_t = \beta_1 \cdot m_{t-1} + (1-\beta_1) \cdot g(t)$$

其中，m_t 是时刻 t 的一阶矩估计，$g(t)$ 是时刻 t 的梯度，β_1 是梯度的衰减率。

华清远见 HQYJ.COM

$$\text{二阶矩估计：} v_t = \beta_2 \cdot v_{t-1} + (1-\beta_2) \cdot g(t) \cdot g(t)$$

其中，v_t 是时刻 t 的二阶矩估计， $g(t)$ 是时刻 t 的梯度，β_2 是梯度平方的衰减率。

一阶矩估计是对梯度的指数加权移动平均，二阶矩估计是对梯度平方的指数加权移动平均，β 是动量和梯度平方的衰减系数（通常取值0到1之间），通常情况下：$\beta_1 = 0.9$，$\beta_2 = 0.999$ 。

2. **估计的是否准确?**

以二阶矩估计为例：

在计算指数加权移动平均时，初始化 $v_0 = 0$，那么$v_1 = 0.999 \cdot v_0 + 0.001 \cdot g_1^2$，得到 $v_1 = 0.001 \cdot g_1^2$，显然得到的 v_1 会小很多，导致估计的不准确，以此类推：

根据：$v_2 = 0.999 \cdot v_1 + 0.001 \cdot g_2^2$，把 v_1 带入后，
得到：$v_2 = 0.999 \cdot 0.001 \cdot g_1^2 + 0.001 \cdot g_2^2$，导致 v_2 远小于 g_1 和 g_2，所以 v_2 并不能很好的估计出前两次训练的梯度。

所以这个估计是有偏差的。

3. **偏差校正引入**

以二阶矩估计距离，在初始迭代时，由于 v_{t-1} 为初始值（通常为0），二阶矩估计 v_t 主要受到当前梯度 $g(W_t)$ 的影响。

如下：

在第一次优化时：

$$v_1 = (1-\beta_2) \cdot g_1^2$$

在第二次优化时：

$$\begin{aligned} v_2 &= \beta_2 \cdot v_1 + (1-\beta_2) \cdot g_2^2 \\ &= \beta_2 \cdot (1-\beta_2) g_1^2 + (1-\beta_2) g_2^2 \\ &= (1-\beta_2)(\beta_2 g_1^2 + g_2^2) \\ &= (1-\beta_2)(\beta_2^{2-1} g_1^2 + \beta_2^{2-2} g_2^2) \\ &= (1-\beta_2) \sum_{i=1}^{2} \beta_2^{2-i} g_i^2 \end{aligned}$$

所以在第t次优化时，得到：

$$v_t = (1-\beta_2) \sum_{i=1}^{t} \beta_2^{t-i} g_i^2$$

修正方案：

华清远见|元宇宙实验中心 yyzlab.com.cn

1. 修正误差方案解释方法1（几何级数的概念，较简单的高中知识）

由于$\sum_{i=1}^{t}\beta_2^{t-i}$是一个**几何级数**（几何级数是数学类名词，表示[等比数列]的前n项和，又称为等比级数），其和小于1，因此v_t会低估实际的梯度平均值。为了修正这种低估，我们需要除以这个级数的和（等比数列的求和公式）：

我们通过简单的带入t=3可以知道：

$\sum_{i=1}^{t}\beta_2^{t-i}=\sum_{i=0}^{t-1}\beta_2^{i}$

相比之下，$\sum_{i=0}^{t-1}\beta_2^{i}$更好求解它的级数的和。

$\text{几何级数和}=\sum_{i=0}^{t-1}\beta_2^{i}=\frac{1-\beta_2^t}{1-\beta_2}$

所以，偏差修正后的一阶矩$\hat{v}_t$为：

$\hat{v}_t=\frac{v_t}{1-\beta_2^t}$

> tips：
> 等比数列求和公式是一种用来计算等比数列前n项之和的公式。等比数列是一种从第二项起，每一项与它的前一项的比等于同一个常数的数列，这个常数叫做等比数列的公比。例如，2, 4, 8, 16, ...就是一个等比数列，其公比为2。
> 利用这个关系，我们可以推导出等比数列求和公式如下：
> $S_n=a_1(1-q^n)/(1-q)$
> 其中，a_n表示第n项，a_1表示首项，q表示公比，S_n表示前n项之和。

2. 修正误差方案解释方法2（数学期望的概念，较复杂的高中知识）

可以得到在t次迭代中，指数加权移动平均中的数学期望是：$E[v_t]$，实际上需要的是梯度的二阶矩估计，即真实的二阶矩为：$E[g_t^2]$，因此可以得到两者之间是有偏的，需要纠正，对原式左右都进行数学期望求解：

$$
\begin{aligned}
E[v_t] &= E\left[(1-\beta_2)\sum_{i=1}^{t}\beta_2^{t-i}\cdot g_i^2\right] \\
&= (1-\beta_2)(1+\beta_2^1+\beta_2^2+\ldots+\beta_2^{t-1})E(g_i^2)+\xi \\
&= (1-\beta_2)(\frac{1-(\beta_2)^t}{1-\beta_2})E(g_i^2)+\xi \\
&= (1-\beta_2^t)E(g_i^2)+\xi
\end{aligned}
$$

ξ 很小，可以忽略，因此，要得到 $E[g_t^2]$，就要去掉前面的系数，即 $(1-\beta_2^t)$。

一阶矩估计的偏差修正：

通过偏差修正（bias correction），即通过除以 $1-\beta_1^t$ 对 m_t 进行调整，其中 t 是迭代次数，在 t 较小时对 m_t 进行调整，有助于减小动量估计的偏差。

一阶矩估计偏差修正方案为：

$$\widehat{m_t} = \frac{m_t}{1-\beta_1^t}$$

二阶矩估计的偏差修正：

通过偏差修正，即通过除以 $1-\beta_2^t$，在 t 较小时对 v_t 进行调整，有助于减小梯度平方估计的偏差。

二阶矩估计偏差修正方案为：

$$\widehat{v_t} = \frac{v_t}{1-\beta_2^t}$$

其中：t代表其t次方，所以刚开始训练时，通过除以就可以很好修正学习速率，当训练多轮时，分母部分也接近1，又回到了原始方程。

3. **Adam更新方案**

更新方案为：

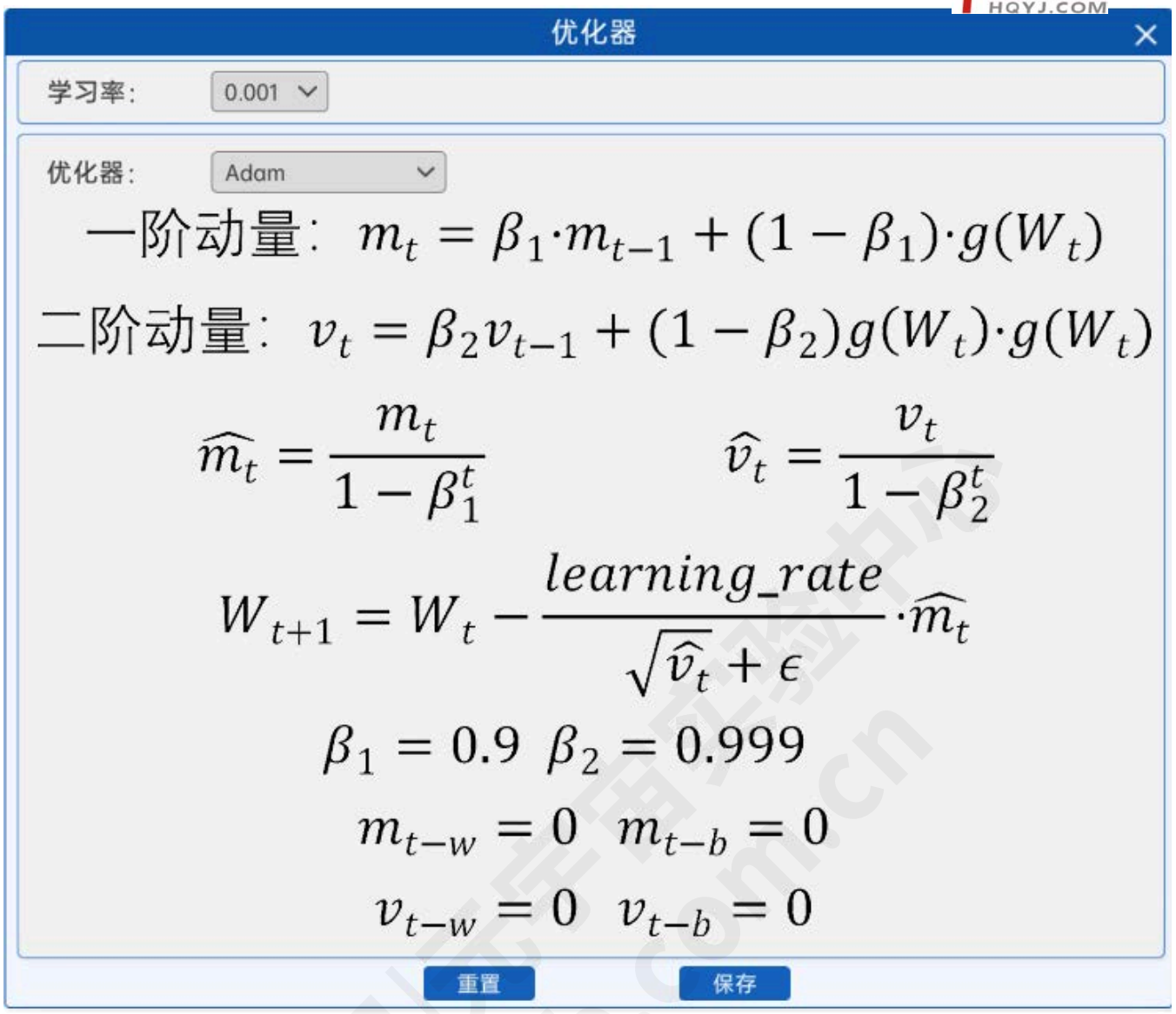
$$W_{t+1} = W_t - \frac{\alpha}{\sqrt{\hat{v}_t}+\epsilon} \cdot \widehat{m}_t$$

其中，ϵ 是为了防止分母为0设置的，一般为10^{-7}。

Adam能够自动调整每个参数的学习率，并且在训练过程中能够不断调整学习率，这个算法对于训练大型深度学习模型非常有用，并且它能够很好地处理噪声和非平稳的训练数据。

如下图所示：

华清远见 HQYJ.COM
华清远见|元宇宙实验中心 yyzlab.com.cn



> 使用Adam生成代码，查看其内部运行规则。

1.5 开始迭代

通过“开始迭代”组件来控制模型的训练次数，不同的是，本实验的“开始迭代”组件中多了一个关于“batch_size”的选项，如下图所示：

batch_size是指每次训练时使用的样本数量，是一个非常重要的超参数，它会影响模型的训练速度和准确性。当batch_size = 1时，如果输入的样本数量是10个，那么就会在一个迭代次数里更新10除以1次w和b，这个的迭代次数就是epoch，使用小的batch_size有几个潜在的好处：

内存效率：使用小的batch_size意味着每次只需加载少量的训练数据到内存中。这在处理大型数据集时特别重要，因为它可以减少内存的需求，使得模型能够适应有限的系统资源。

参数更新频率：使用小的batch_size可以增加参数的更新频率。由于每个小批量的数据都会产生一次参数更新，使用小的batch_size可以使得参数更频繁地更新，从而加快模型的收敛速度。

噪声鲁棒性：小的batch_size可以引入一定程度的随机性和噪声，从而提高模型的鲁棒性。每个小批量的数据可能具有不同的特征和噪声，这可以帮助模型更好地适应不同类型的样本。

训练稳定性：对于某些复杂的模型或具有较大参数空间的模型，使用小的batch_size可以提高训练的稳定性。小批量数据的使用可以减少参数更新的方差，并且可能有助于避免过拟合。

然而，小的batch_size也可能存在一些挑战和限制：

训练速度：使用小的batch_size可能导致训练过程变慢。由于参数更新频率增加，每个小批量的计算成本也增加，可能导致训练时间延长。

精确度：较小的batch_size可能导致训练过程中的梯度估计不准确。较小的样本集可能不足以代表整个数据分布，从而影响梯度的估计和参数的更新方向。

学习率调整：使用小的batch_size通常需要更小的学习率来保持稳定性。由于参数更新更频繁，较大的学习率可能导致训练过程不稳定。

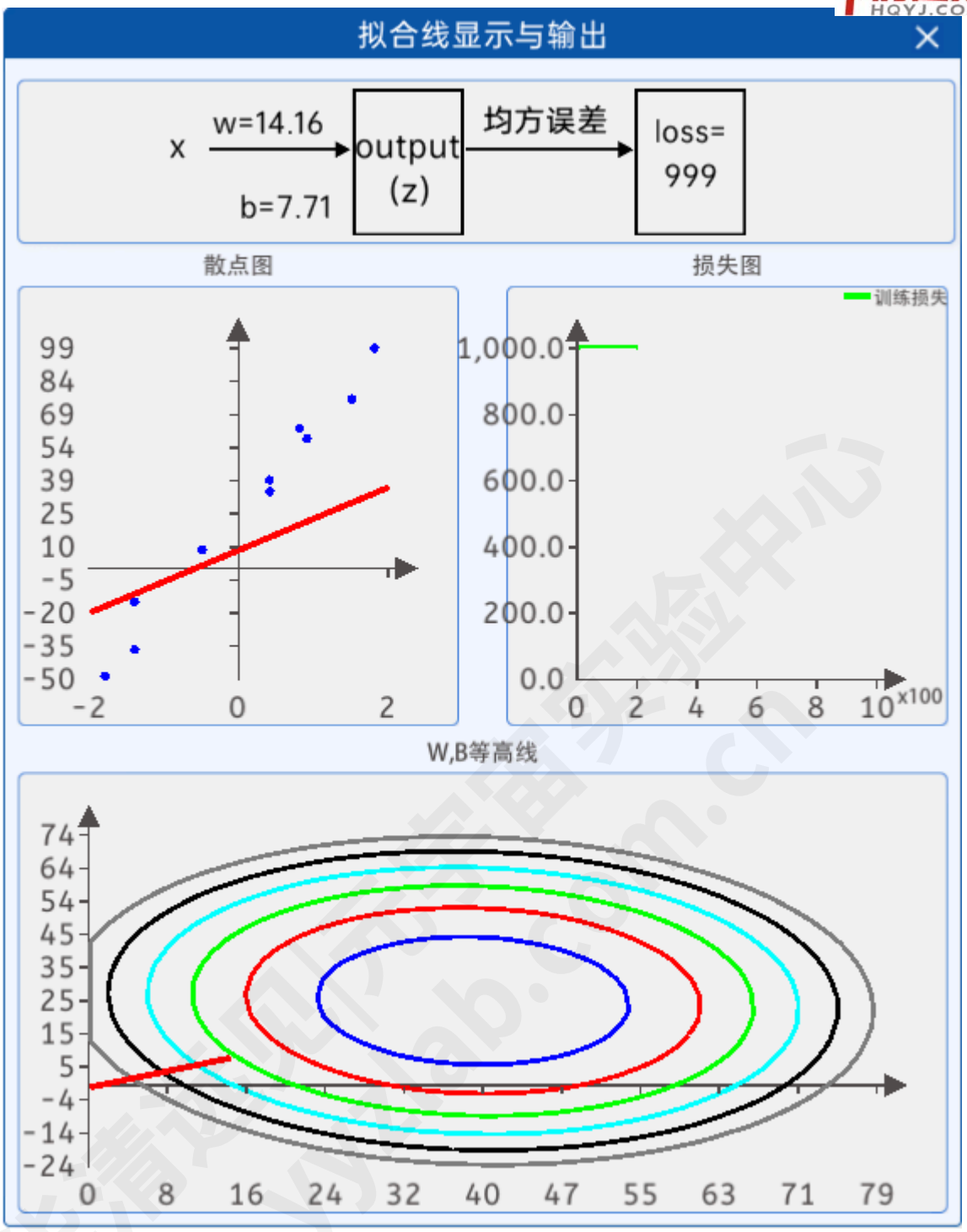
综上所述，选择合适的batch_size需要考虑到数据集的大小、系统资源、模型复杂度和训练速度等因素。通常，选择一个适度的batch_size可以在训练效率和模型性能之间取得平衡。最佳的batch_size可能需要通过实验和调整来确定，以满足特定问题和任务的需求。

1.6 显示频率设置

通过设置显示频率，查看迭代过程中各个参数的变化及拟合线的形状。

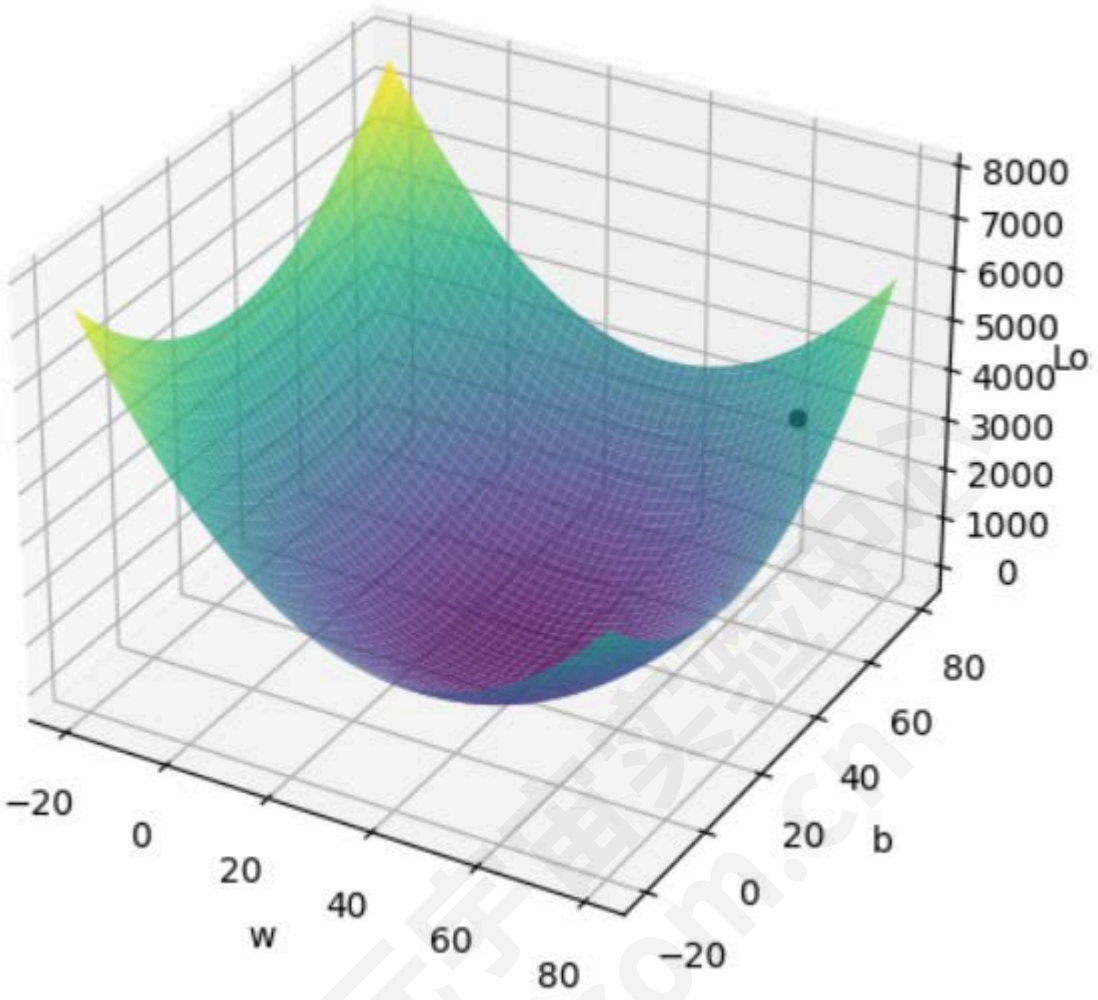
1.7 拟合线显示与输出

迭代结果通过“拟合线显示与输出”组件来查看，如下图所示。



它会根据迭代次数实时显示w、b以及损失值，并且第一幅图是根据实时的w、b值进行绘制的，第二幅图是以实时的损失值为纵坐标、迭代次数为横坐标进行绘制的，第三幅图则是显示当前梯度值所在的等高线位置。

为什么要绘制等高线呢？如下图所示：



其实loss关于w和b的图是一张三维图，在更新参数时不仅要计算w方向上的梯度，还要计算b方向上的梯度。于是就出现了这张三维立体图，而我们的实验中则是画出了它的等高线图，通过等高线图也能感受到损失值的变化情况。

2. 编程实例与步骤

需要默认打开“华清远见人工智能虚拟仿真”和“华清远见人工智能虚拟仿真本地服务管理平台”。

2.1 （可选）使用“华清远见人工智能虚拟仿真本地服务管理平台”启动服务。

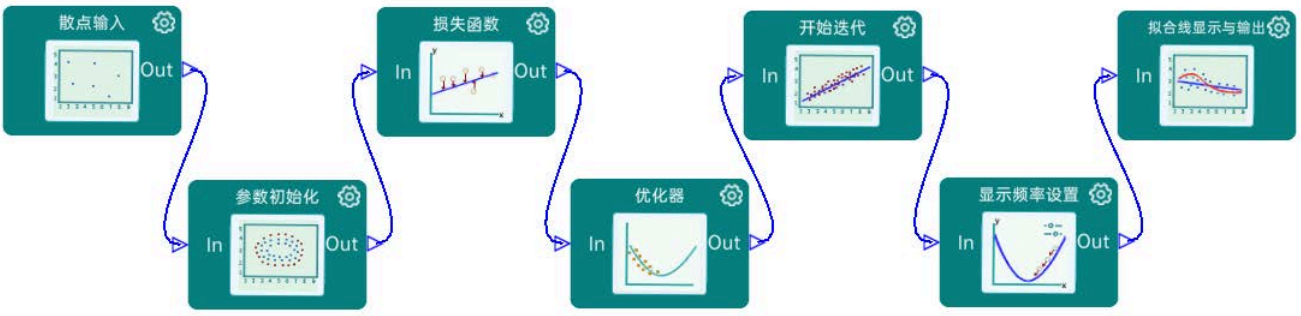
服务启动一次之后就不需要再次启动了。

单击优化器与优化方法实验进入

如果之前进入了某个实验，可以在实验面板的右上角点击“返回”回到主页即可。

2.2 拖出组件并连线

根据上述实验原理，将我们的逻辑抽象成组件之后，连线如下图所示。



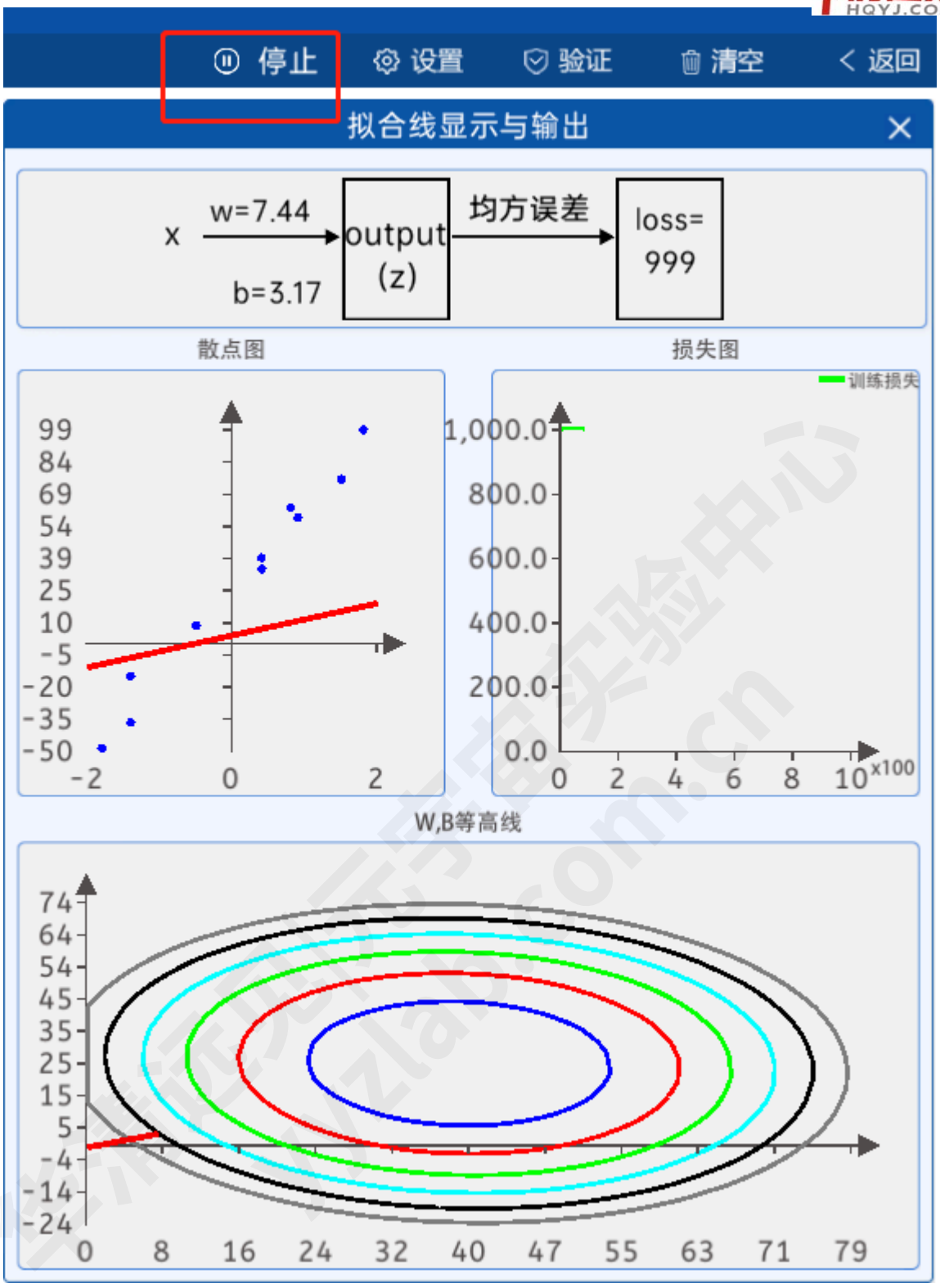
2.3 验证

点击验证，如显示校验成功，即代表逻辑无误，验证按钮与校验成功界面如下图所示。

2.4 运行

点击运行，可以实现功能运行，并显示结果输出，如下图所示。

华清远见元宇宙实验中心
yyzlab.com.cn



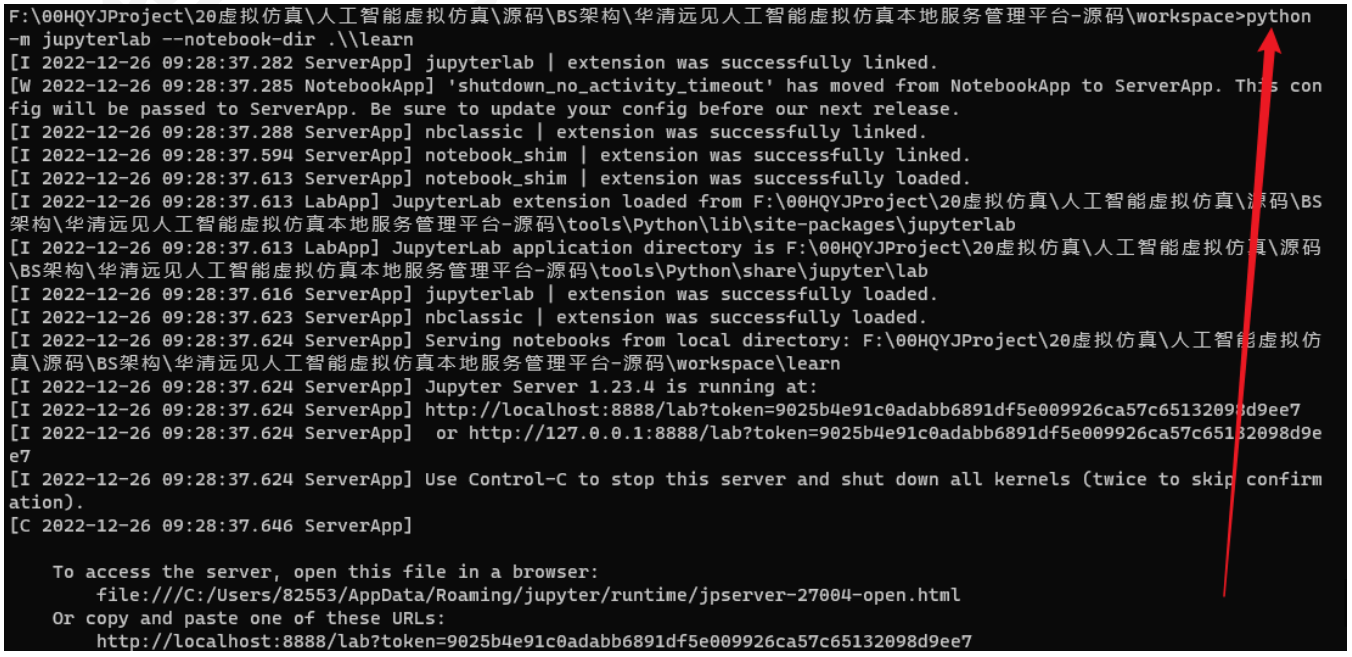
2.5 代码生成

点击界面左下角的生成代码按钮，可生成本实验的Python代码，生成代码按钮如下图所示。

点击后可以查看该实验代码，代码会根据界面所选参数进行适配，点击复制可以将其复制，并粘贴在其它地方。

这里我们可以粘贴到软件的Jupyter中运行，“华清远见人工智能虚拟仿真本地服务管理平台”打开Jupyter，第一次打开Jupyter需要点击“打开CMD命令”，打开Jupyter后无需再次打开CMD，如下图所示。

在弹出的CMD命令行中，输入python -m jupyterlab --notebook-dir .\\learn，点击回车，浏览器会自动跳转打开Jupyter，如下图所示。

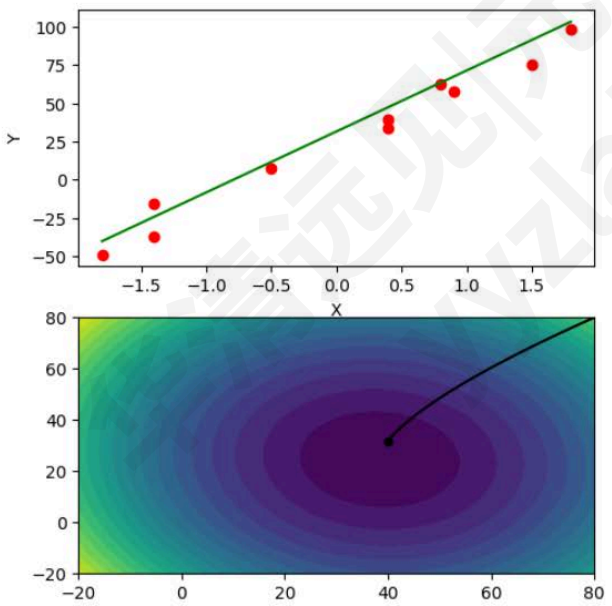
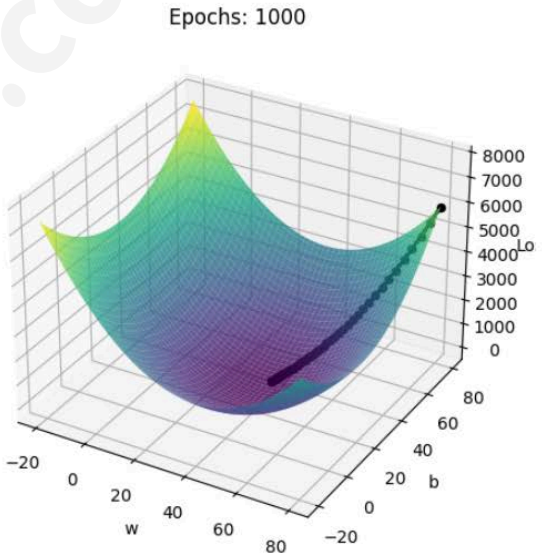


如果没有自动跳转和打开浏览器，请输入下图中圈出的内容，每个人的该内容不一致，请复制自己的 弹出的。

```
F:\00HQYJProject\20虚拟仿真\人工智能虚拟仿真\源码\BS架构\华清远见人工智能虚拟仿真本地服务管理平台-源码\workspace>python
-m jupyterlab --notebook-dir .\\learn
[I 2022-12-26 09:28:37.282 ServerApp] jupyterlab | extension was successfully linked.
[W 2022-12-26 09:28:37.285 NotebookApp] 'shutdown_no_activity_timeout' has moved from NotebookApp to ServerApp. This con
fig will be passed to ServerApp. Be sure to update your config before our next release.
[I 2022-12-26 09:28:37.288 ServerApp] nbclassic | extension was successfully linked.
[I 2022-12-26 09:28:37.594 ServerApp] notebook_shim | extension was successfully linked.
[I 2022-12-26 09:28:37.613 ServerApp] notebook_shim | extension was successfully loaded.
[I 2022-12-26 09:28:37.613 LabApp] JupyterLab extension loaded from F:\00HQYJProject\20虚拟仿真\人工智能虚拟仿真\源码\BS
架构\华清远见人工智能虚拟仿真本地服务管理平台-源码\tools\Python\lib\site-packages\jupyterlab
[I 2022-12-26 09:28:37.613 LabApp] JupyterLab application directory is F:\00HQYJProject\20虚拟仿真\人工智能虚拟仿真\源码
\BS架构\华清远见人工智能虚拟仿真本地服务管理平台-源码\tools\Python\share\jupyter\lab
[I 2022-12-26 09:28:37.616 ServerApp] jupyterlab | extension was successfully loaded.
[I 2022-12-26 09:28:37.623 ServerApp] nbclassic | extension was successfully loaded.
[I 2022-12-26 09:28:37.624 ServerApp] Serving notebooks from local directory: F:\00HQYJProject\20虚拟仿真\人工智能虚拟仿
真\源码\BS架构\华清远见人工智能虚拟仿真本地服务管理平台-源码\workspace\learn
[I 2022-12-26 09:28:37.624 ServerApp] Jupyter Server 1.23.4 is running at:
[I 2022-12-26 09:28:37.624 ServerApp] http://localhost:8888/lab?token=9025b4e91c0adabb6891df5e009926ca57c65132098d9ee7
[I 2022-12-26 09:28:37.624 ServerApp]  or http://127.0.0.1:8888/lab?token=9025b4e91c0adabb6891df5e009926ca57c65132098d9e
e7
[I 2022-12-26 09:28:37.624 ServerApp] Use Control-C to stop this server and shut down all kernels (twice to skip confirm
ation).
[C 2022-12-26 09:28:37.646 ServerApp]

    To access the server, open this file in a browser:
        file:///C:/Users/82553/AppData/Roaming/jupyter/runtime/jpserver-27004-open.html
    Or copy and paste one of these URLs:
        http://localhost:8888/lab?token=9025b4e91c0adabb6891df5e009926ca57c65132098d9ee7
```

将代码输入到Jupyter中，点击运行可以查看结果。

3. 实验现象

实验结果为：